



Exercises session 1

Turing Machines and Time Complexity

1 Definitions

Definition 1.1 (Turing Machine) We define a (deterministic) Turing machine as a 7-tuple: $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ where

- Q is a finite set of states
- Σ is the input alphabet, where $\vdash \notin \Sigma$
- Γ is the tape alphabet, where $\Sigma \cup \{\vdash\} \subseteq \Gamma$
- $q_0 \in Q$ is the initial state
- $q_{\text{accept}} \in Q$ is the accepting state
- $q_{\text{reject}} \in Q$ is the rejecting state ($q_{\text{accept}} \neq q_{\text{reject}}$)
- δ is a partial function:

$$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{\leftarrow, \square, \rightarrow\}$$

δ is called the transition function

Definition 1.2 (Time complexity) The time complexity of a TM M in the worst case is defined by the function t_M :

$$t_M(n) = \max\{ \text{time}_M(w) \mid w \in \Sigma^* \text{ and } |w| = n \}.$$

where $\text{time}_M(w)$ is the number of steps that M executes on input $w \in \Sigma^*$

Definition 1.3 (Asymptotic complexity notation) We recall the following notation:

- **Big-O ($\leq$):** $f(n) = O(g(n))$ if:

$$\exists c, n_0 \text{ s.t. } \forall n \geq n_0 \quad f(n) \leq c \cdot g(n)$$

- **Small-o ($<$):** $f(n) = o(g(n))$ if:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0; \text{ that is,}$$

$$\forall c > 0 \exists n_0 \text{ s.t. } \forall n \geq n_0 \quad f(n) < c \cdot g(n)$$

- **Big-Omega ($\geq$):** $f(n) = \Omega(g(n))$ if:

$$g(n) = O(f(n)), \text{ that is}$$

$$\exists c, n_0 \text{ s.t. } \forall n \geq n_0 \quad g(n) \leq c \cdot f(n), \text{ that is}$$

$$\exists d, n_0 \text{ s.t. } \forall n \geq n_0 \quad f(n) \geq d \cdot g(n)$$

2 Session exercises

1. Let's consider a new definition of a turing machine, where in each step the head has to move either to the right or to the left (it cannot stay in the same place). Show that this can simulate any regular deterministic turing machine. What can we say about the running time?
2. Define a turing machine $\mathcal{M}$ that decides the following language

$$L = \{0^k 1^k : k \in \mathbb{N}\}$$

- (a) Implement such turing machine in <https://turingmachinesimulator.com/>
 - (b) What is the time complexity of $\mathcal{M}$?
 - (c) Can we do it better than $O(n^2)$?
3. Prove if the following are true or false
 - (a) $2^{n+1} = O(2^n)$
 - (b) if $f(n) = O(g(n))$ and $g(n) = O(h(n))$ then $f(n) = O(h(n))$
 - (c) there exists $f(n)$ and $g(n)$ such that $f(n) = o(g(n))$ and $f(n) = \Omega(g(n))$
 - (d) for every pair of functions f and g , either $f(n) = O(g(n))$ or $g(n) = O(f(n))$

3 Proposed exercises

1. Create a program in <https://turingmachinesimulator.com/> that decides the following language

$$L = \{ w \cdot \# \cdot w^{-1} \mid w \in \{0, 1\}^* \text{ and } |w| \leq 4 \}.$$

Here w^{-1} denotes the reverse of the word w , and $|w|$ denotes the length of w . Provide a big-O analysis of your turing machine.

2. Prove if the following are true or false

• $2^{2n} = O(2^n) \rightarrow \text{FALSE.}$ $2^{2n} = 2^n \cdot 2^n$
 • if $f(n) = O(g(n))$ and $g(n) = O(f(n))$ then $f(n) = g(n) \rightarrow \text{FALSE.}$
 • if $f(n) = O(n)$, then $2^{f(n)} = O(2^n) \rightarrow \text{FALSE.}$

$f(m) = 2m$ $f(m) = 2m$
 $g(m) = 3m$

2

Solutions

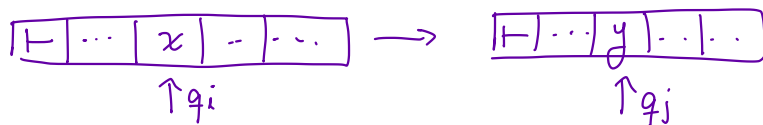
①

1. Let's consider a new definition of a Turing machine, where in each step the head has to move either to the right or to the left (it cannot stay in the same place). Show that this can simulate any regular deterministic Turing machine. What can we say about the running time?

the intuition is to simulate the "stay" step with a "backward-forward" pair of steps.

this is, we will need an extra state; say q_i^j , for each state q_i and then;

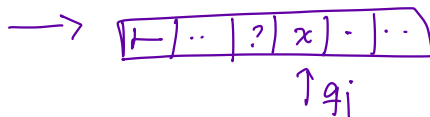
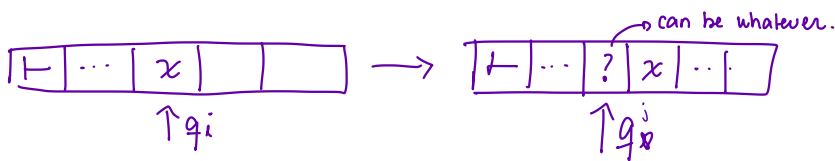
$$(q_i, x) \rightarrow (q_j, y, \square)$$



$\Downarrow$ if you can't stay.

$$(q_i, x) \rightarrow (q_i^j, y, \leftarrow)$$

$$(q_i^j, ?) \rightarrow (q_j, -, \rightarrow)$$



now formally, given a standard TM

$$\mathcal{M} = (Q, \Sigma, \Gamma, q_0, q_a, q_r, \delta)$$

we build a "cut-stay" - TM $\mathcal{M}'$

$$\mathcal{M}' = (Q', \Sigma, \Gamma, q_0, q_a, q_r, \delta')$$

$$\text{where } Q' = Q \cup \{q_0^j : j \in \{1, 2, \dots, |\Sigma|\}\}$$

$$\text{and } \delta' : Q' \times \Gamma \longrightarrow Q' \times \Gamma \times \{\leftarrow, \rightarrow\}$$

now for every $(q, x) \in Q \times \Gamma$ such that

$$\delta(q, x) = (q', x', A) \quad A \in \{\leftarrow, \rightarrow\} \quad \begin{matrix} q' \in Q \\ x' \in \Gamma \end{matrix}$$

then

$$\delta'(q, x) = \delta(q, x)$$

(keep the forward and backward transitions)

$$\text{and for } (q, x) \text{ s.t. } \delta(q, x) = (q_j, x', \square)$$

we do

$$\delta'(q, x) = (q_0^j, x', \leftarrow)$$

and

$$\delta'(q_j^i, y) = (q_j, y, \rightarrow)$$

Finally, we need to prove that

$$L(\mathcal{M}) = L(\mathcal{M}') \quad \left(\begin{array}{l} \text{the TM's accept the} \\ \text{same language} \end{array} \right)$$

First

$$\mathcal{L}(\mathcal{M}) \subseteq \mathcal{L}(\mathcal{M}')$$

take $w \in \mathcal{L}(\mathcal{M})$ and we want to prove that $w \in \mathcal{L}(\mathcal{M}')$

if $w \in \mathcal{L}(\mathcal{M})$ then there's a sequence of configurations

$$C_1 \dots C_k$$

$$\text{st. } C_1 = q_0 w$$

$$C_k = w_1 q_2 w_2 \quad \text{with } w_1 \cdot w_2 = w$$

we only need to care about configurations that look like this.

$$\begin{array}{l} C_i = w_1 \cdot q_i \cdot w_2 \\ C_{i+1} = w_1 \cdot q_j \cdot w_2 \end{array} \quad \left. \begin{array}{l} \\ \end{array} \right\} \begin{array}{l} \text{the head stays in the} \\ \text{some place} \end{array}$$

if such pair of conf exists in the sequence, then in $\mathcal{M}'$ we can do the following.

$$\begin{array}{l} \text{let } w_1 = l_1 \dots l_k \\ w_2 = l_{k+1} \dots l_n \end{array} \quad \left\{ \begin{array}{l} w = l_1 \dots l_n \\ l_i \in \Sigma \end{array} \right.$$

$$C_i = l_1 \dots l_k q_i l_{k+1} \dots l_n$$

$$C_{i+1} = l_1 \dots l_{k-1} q_k^j l_k \dots l_n$$

$$C_{i+2} = l_1 \dots l_k q_j l_{k+1} \dots l_n$$

as we kept all other transitions, by replacing C_i, C_{i+1} by C_i, C_{i+2}, C_{i+2} , we will have an accepting configuration for $\mathcal{M}'$.

second

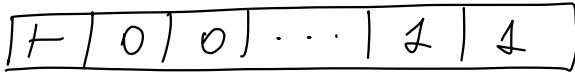
$$\mathcal{L}(\mathcal{M}') \subseteq \mathcal{L}(\mathcal{M})$$

(left as an exercise to you )

2. Define a turing machine $\mathcal{M}$ that decides the following language

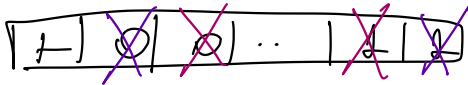
$$L = \{0^k 1^k : k \in \mathbb{N}\}$$

- (a) Implement such turing machine in <https://turingmachinesimulator.com/>
- (b) What is the time complexity of $\mathcal{M}$?
- (c) Can we do it better than $O(n^2)$?



① scan the tape to see if there are no 0's after a 1.

② go matching 0's with ones



③ accept if tape is empty, reject otherwise

↳ each scan $\rightarrow O(n)$

↳ in each scan we remove 2 letters.

how many scans? $\Theta(n)$

$\Rightarrow$ runtime $\Theta(n^2)$

we can do this in $O(m \log m)$

① do a scan to see if there's no 0's after a 1

② while tape is not empty:

- cross every other 0, starting from the first one
- cross every other 1, starting from the first one
- if we have an odd number of letters left, reflect.
- if is even, go on.

each scan $O(m)$
scans $O(\log m)$
at each step we halve the size of the word.

③ accept $\rightarrow O(m \log m)$.

Why this work? because its checking if the binary representation of the numbers of 0's and 1's is the same.

e.g. $0^{13} 1^{12}$

$13 = 1101$
 $12 = 1100$

	:2	rest
13	6	1
	3	0
	1	1
	1	1

	:2	rest
12	6	0
	3	0
	1	1
	1	1

by the end, you will have one 0's and two 1's, so the sum will be odd

3. Prove if the following are true or false

(a) $2^{n+1} = O(2^n)$

(b) if $f(n) = O(g(n))$ and $g(n) = O(h(n))$ then $f(n) = O(h(n))$

(c) there exists $f(n)$ and $g(n)$ such that $f(n) = o(g(n))$ and $f(n) = \Omega(g(n))$

(d) for every pair of functions f and g , either $f(n) = O(g(n))$ or $g(n) = O(f(n))$

(a) $2^{m+1} = 2^m \cdot 2$

we need to prove

$(\exists c, n_0) \forall n \geq n_0$

$\exists c, n_0 \text{ s.t. } \forall n \geq n_0 \quad f(n) \leq c \cdot g(n)$

$c = 3 \quad n = 1$

$$2^{m+1} \leq 3 \cdot 2^m$$

$$\cancel{2 \cdot 2^m} \leq \cancel{3 \cdot 2^m} \quad \forall m.$$

(b) $\exists c_1, n_1 \quad \forall$

$$f(m) \leq c_1 \cdot g(m) \quad \forall m \geq n_1$$

$$g(m) \leq c_2 \cdot h(m) \quad \forall m \geq n_2$$

$$n' = \max(n_1, n_2) \quad \begin{array}{l} f(m) \leq c_1 \cdot g(m) \\ g(m) \leq c_2 \cdot h(m) \end{array}$$

$$f(m) \leq c_1 \cdot c_2 \cdot h(m) \quad \therefore f(m) = O(h(m))$$

(c) let's assume that it does, then

$$f(m) < c \cdot g(m) \quad \forall m \geq m_1$$

$$f(m) \geq d \cdot g(m) \quad \forall m \geq m_2$$

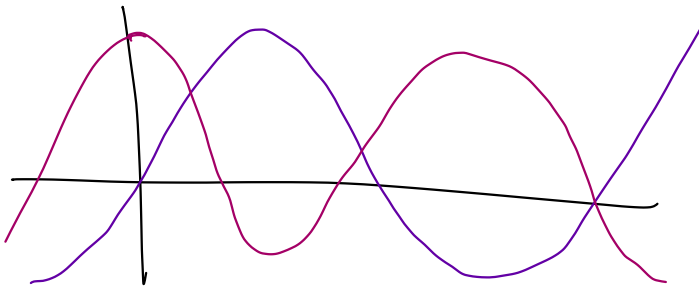
if we take $m = \max\{m_1, m_2\}$

$$f(m) < c \cdot g(m) \Rightarrow \frac{f(m)}{c} < g(m)$$

$$d \cdot g(m) \leq f(m) \Rightarrow g(m) \leq \frac{f(m)}{d}$$



(d) we can do $\sin(x)$ and $\cos(x)$



$$\sin(x) \notin \Theta(\cos(x))$$

$$\cos(x) \notin \Theta(\sin(x))$$